

Software Project Management (SPM) (CSC415)

Course Instructor
Er. ANUJ GHIMIRE

DISCLAIMER

- *This document does not claim any originality and cannot be used as a substitute for prescribed textbooks.*
- *The information presented here is merely a collection from various sources as well as freely available material from internet and textbooks.*
- *The ownership of the information lies with the respective authors or institutions.*



Chapter 8: Software Quality Assurance and Testing

Testing

- Software quality of the system means, system having robust, precise, and maintainable.
- Testing is an iterative process that is carried out in conjunction with implementation.
- It is the most common way of checking that it meets its specification and does what the customer wants.
- It is an investigation conducted to provide stakeholders with information about the quality of the product or service under test.

Testing

- Testing is a critical element of software quality assurance and represents the ultimate review of specification, design, and code generation.
- Testing is done to improve *quality, to verify and validate, for reliability estimation* etc.

Principles of Testing

- ***Testing***
 - The execution of a program to find its faults
- ***Verification***
 - The process of proving the programs correctness.
- ***Validation***
 - The process of finding errors by executing the program in a real environment
- ***Debugging***
 - Diagnosing the error and correct it

Software Testing Principles

- A common view of testing is that all untested code has a roughly equal probability of containing defects.
- The objective should therefore be to remove as many as defects as possible before test since the quality improvement potential of testing is limited.
- An examination of even relatively simple programs demonstrates that exhaustive testing is generally impossible.
 - *If a program were to analyze a string of only ten alphabetic characters, there would be **26^{10} possible combinations**. Testing one condition every microsecond would take four and a half million years.*
- Thus test design reduces to a small subset of conditions that will reveal the characteristics of the program.

Software Testing Principles

- To remove as many defects as possible before test since the quality improvement potential of testing is limited
 - *All tests should be tractable to customers requirements.*
 - *Tests should be planned long before testing begins.*
 - *Testing should begin “**in the small**” and progress toward testing in the large.*
 - *80% of all errors uncovered during testing will likely be tractable to 20% of components. That's why testing should be done of all region components.*
 - *Testing should be done early to minimize errors.*

Objective of Testing

- To demonstrate to the developer and the customer that the software meets its requirements.
- Finding defects which may get created by the programmer. while developing the software.
- Gaining confidence in a providing information about the level of quality.
- To make sure that the end result meets the business and user requirements.
- To ensure that it satisfies Business Requirement Specification (BRS) and System Requirement Specification (SRS).
- To gain the confidence of the customers by providing them a quality product.

Test Case

- A rich variety of test case design methods, which provide the developer with a systematic approach to testing.
- A test case in software engineering is a set of conditions or variables under which a tester will determine whether an application or software system is working correctly.
- These methods provide a mechanism that can help to ensure the completeness of tests and provide the highest likelihood for uncovering errors in software.
- The mechanism for determining whether a software program or system has passed or failed such a test is known as a **test oracle**.

Test Plan

- Test plan is a document detailing a systematic approach to testing a system such as a machine or software.
- The plan typically contains a detailed understanding of what the eventual workflow will be.
- A test plan documents the strategy that will be used to verify and ensure that a product or system meets its design specifications and other requirements.
- A test plan is usually prepared by or with significant input from Test Engineers.

Test Plan

- A test plan will include the following.
 - *Introduction to the Test Plan document*
 - *Assumptions when testing the application*
 - *List of test cases included in Testing the application*
 - *List of features to be tested*
 - *What sort of Approach to use when testing the software*
 - *List of Deliverables that need to be tested*
 - *The resources allocated for testing the application*
 - *Any Risks involved during the testing process*
 - *A Schedule of tasks and milestones as testing is started*

Types of Testing

- **Manual Testing:**

- *This type includes the testing of the Software manually i.e. without using any automated tool or any script.*
- *In this type the tester takes over the role of an end user and test the Software to identify any un-expected behavior or bug.*

- **Automation Testing:**

- *Automation testing which is also known as Test Automation, is when the tester writes scripts and uses another software to test the software.*
- *This process involves automation of a manual process.*
- *Automation Testing is used to re-run the test scenarios that were performed manually, quickly and repeatedly.*

Level of Testing

- Testing levels are the procedure for finding the missing areas and avoiding overlapping and repetition between the development life cycle stages.
- We have already seen the various phases such as Requirement collection, designing, coding testing, deployment, and maintenance of SDLC (Software Development Life Cycle).
- In order to test any application, we need to go through all the above phases of SDLC.
- Like SDLC, we have multiple levels of testing, which help us maintain the quality of the software.

Level of Testing

- The levels of software testing involve the different methodologies, which can be used while we are performing the software testing.
- In software testing, we have four different levels of testing, which are as discussed below:
 - *Unit Testing*
 - *Integration Testing*
 - *System Testing*
 - *Acceptance Testing*

Unit Testing

- This type of testing uses tests for a single component or a single unit in software testing and this kind of testing is performed by the developer.
- Unit testing is also the first level of functional testing.
- The primary goal of unit testing is to validate the performance of unit components.
- Unit is the smallest testable portion of the system or application.
- The main aim is to test that each component or unit is correct in fulfilling requirements and desired functionality.

Integration Testing

- Integration testing means combining different software modules and phases and testing as a group.
- It is done to ensure that the integrated system is ready for system testing or not, and there are many ways to test how different components of the system function at their interface.
- This type of testing is performed by testers and integration testing finds the data flow from one module to other modules.

System Testing

- System testing is most probably the final test to identify that the system meets the specification and criteria and it evaluates both function and non-functional needs for the testing.
- System testing is allowing to check the system's compliance as per the requirements and all the components of the software are tested as a whole to ensure that the overall product meets the requirements specified.
- It involves load, reliability, performance, and security testing.

System Testing

- System testing is a very important step as the software is almost ready for production in the market.
- Once it is deployed it can be tested in an environment that is very close to the market/user-friendly environment which the user will experience.

Acceptance Testing

- Acceptance testing aims to evaluate whether the system complies with the end-user requirements and if it is ready for deployment.
- The tester will utilize a different method such as pre-written scenarios and test cases to test the software and use the results obtained from these tools to find ways in which the system can be improved.
- Also QA team or testing team can find out how the product will perform when it is installed on the user's system.
- Acceptance testing ranges from easily finding spelling mistakes and cosmetic errors to relatable bugs that could cause a major error in the application.

Testing Strategies

- A strategy for software testing must accommodate low-level tests that are necessary to verify that a small source code segment has been correctly implemented as well as high-level tests that validate major system functions against customer requirements.
- A strategy must provide guidance for the practitioner and a set of milestones for the manager.
- Because the steps of the test strategy occur at a time when deadline pressure begins to rise, progress must be measurable and problems must surface as early as possible.

Testing Strategies

- Testing begins at the component level and works outward toward the integration of the entire computer-based system.
- Different testing techniques are appropriate at different points in time.
- The developer of the software conducts testing and may be assisted by independent test groups for large projects.
- The role of the independent tester is to remove the conflict of interest inherent when the builder is testing his or her own product.

Testing Strategies

- Testing and debugging are different activities.
- Debugging must be accommodated in any testing strategy.
- Make a distinction between verification (are we building the product right?) and validation (are we building the right product?)
- There are three approaches of testing:
 - *Static Test Strategy*
 - *Structural Test Strategy*
 - *Behavioral Test Strategy*

Testing Strategies Approaches

- *Static Test Strategy:*

- It evaluates the quality of a system without actually running the system.
- It looks at portion or system elements to detect problems as early as possible strategy saves time and money earlier.
- This strategy saves time and money since it tries to detect problem earlier.
- Static tests must be performed at the right time.

Testing Strategies Approaches

- ***Structural Test Strategy:***

- Structural tests need to be operated on real devices and the system has to be run in its entirety to find all the bugs.
- It is often run on individual components and interfaces to identify localized error in data flows.

Testing Strategies Approaches

- ***Behavioral Test Strategy:***

- It focuses on how a system acts rather than the mechanism behind its functions.
- It focuses on workflows, configurations, performance, and all elements of the user journey.

Verification & Validation

- Verification refers to the set of tasks that ensure that software correctly implements a specific function.
- Validation refers to a different set of tasks that ensure that the software that has been built is traceable to customer requirements.
- Boehm [Boe81] states this another way:
 - *Verification: "Are we building the product right?"*
 - *Validation: "Are we building the right product?"*

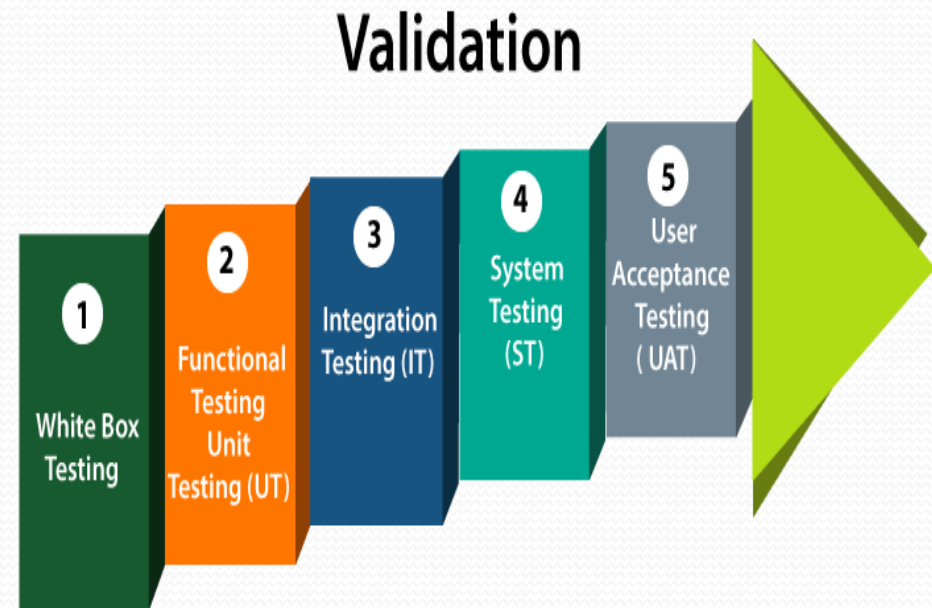
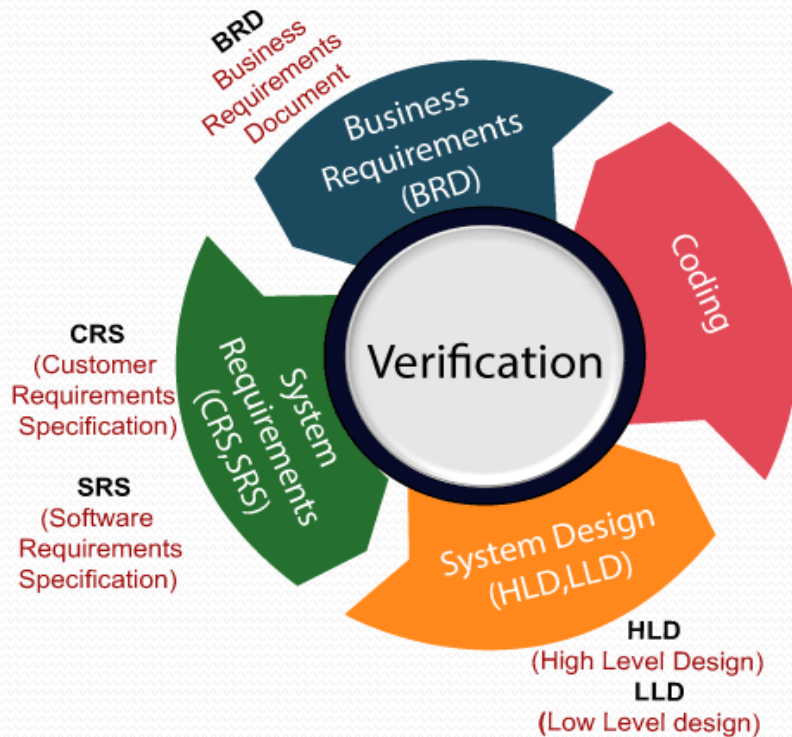
Verification & Validation

- *Verification Testing* includes different activities such as business requirements, system requirements, design review, and code walkthrough while developing a product.
- It is also known as static testing, where we are ensuring that "we are developing the right product or not".
- And it also checks that the developed application fulfilling all the requirements given by the client.

Verification & Validation

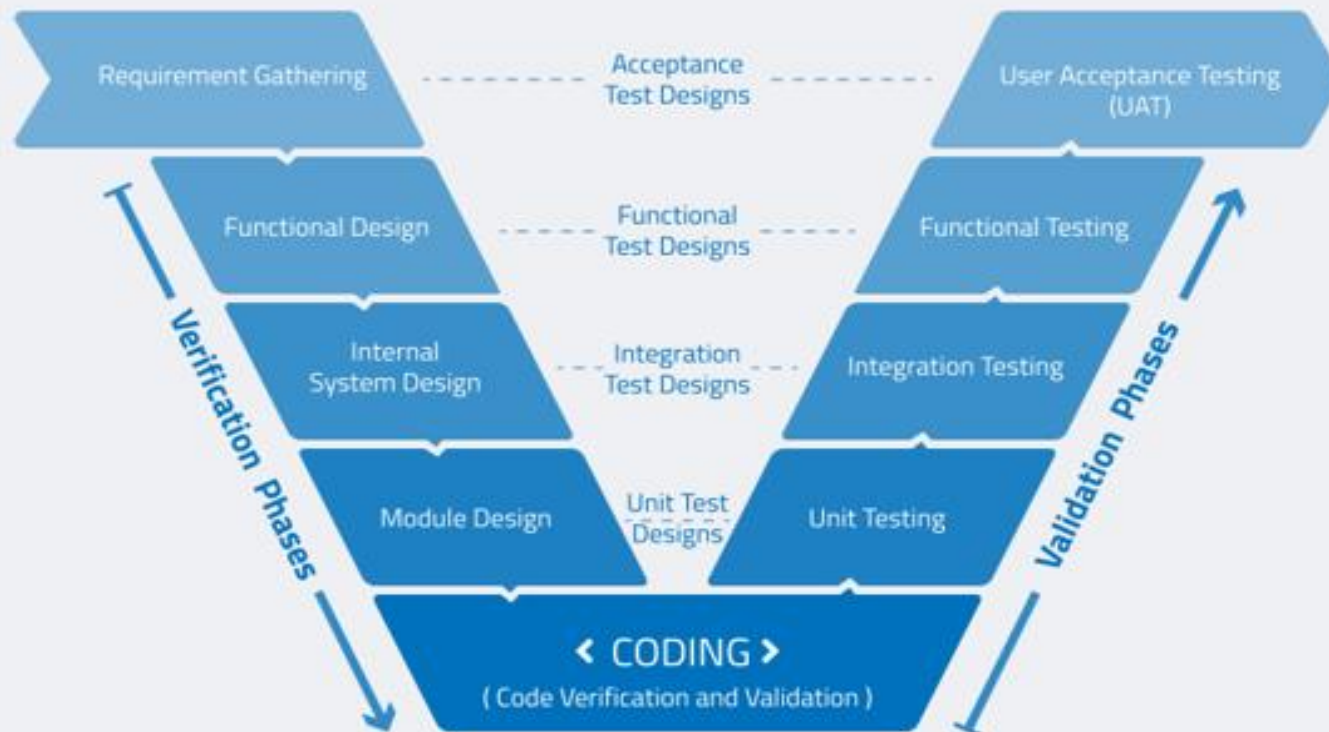
- ***Validation Testing*** is testing where tester performed functional and non-functional testing.
- Here functional testing includes Unit Testing (UT), Integration Testing (IT) and System Testing (ST), and non-functional testing includes User acceptance testing (UAT).
- Validation testing is also known as dynamic testing, where we are ensuring that "we have developed the product right."
- And it also checks that the software meets the business needs of the client.

Verification & Validation



Verification and Validation process are done under the V model of the SDLC

Verification And Validation Model



Software Quality

- Software quality is defined as a field of study and practice that describes the desirable attributes of software products.
- Software quality refers to the degree to which a software product meets its specified requirements and user expectations.
- There are several factors that contribute the software quality including:
 - *Functionality*
 - *Reliability*
 - *Usability*
 - *Efficiency*
 - *Maintainability*

Software Quality

- According to Demin:
 - *“The problem inherent in attempts to define the quality of a product, almost any product, were stated by the master Walter A. Shewhart.*
 - *The difficulty in defining quality is to translate future needs of the user into measurable characteristics, so that a product can be designed and turned out to give satisfaction at a price that the user will pay.*
 - *This is not easy, and as soon as one feels fairly successful in the endeavor, he finds that the needs of the consumer have changed, competitors have moved in, etc.”*

Software Quality

- Quality refers to how it meets non-functional requirements that support the delivery of the functional requirements, such as robustness or maintainability, the degree to which the software was produced correctly.
- There are several methods and techniques for ensuring software quality including:
 - *Testing*
 - *Code Reviews*
 - *Quality Assurance Process.*
- The goal of these method is to identify and fix defects and bugs in the software, improve performance, and enhance user satisfaction.

SEI

- SEI stands for Software Engineering Institute
- The Carnegie Mellon Software Engineering Institute (SEI) is a federally funded research and development center headquartered on the campus of Carnegie Mellon University in Pittsburgh, Pennsylvania, United States.
- It works closely with industry and academia through research collaborations.
- Principal areas of SEI: acquisition, process management, risk, security, software development, and system design.

Mission if SEI

- ***Research***
 - Advancing the science and practice
- ***Collaboration***
 - Bringing together and building on work found in industry, academia, and government
- ***Development and Demonstration***
 - Maturing promising technologies and practices and demonstrating their utility through trial application and prototypes
- ***Transition***
 - Propagating proven technologies and practices through publication, standards and other venues

CMM

- CMM stands for Capability Maturity Model.
- It is a development model created after study of data collected from organizations that contracted with the U.S. Department of Defense.
- The model's aim is to improve existing software-development processes
- The model involves five aspects:
 - *Maturity Levels*
 - *Key Process Areas*
 - *Goals*
 - *Common Features*
 - *Key Practices*

CMM

- ***Maturity Levels***

- It is a 5-level process maturity scale - where the uppermost (5th) level is a notional ideal state where processes would be systematically managed by a combination of process optimization and continuous process improvement.

- ***Key Process Areas***

- Key Process Area identifies a cluster of related activities that, when performed together, achieve a set of goals considered important.

CMM

- ***Goals***

- The goals of a key process area summarize the states that must exist for that key process area to have been implemented in an effective and lasting way.
- The extent to which the goals have been accomplished is an indicator of how much capability the organization has established at that maturity level.
- The goals signify the scope, boundaries, and intent of each key process area.

CMM

- ***Common Features***

- common features include practices that implement and institutionalize a key process area.
- There are five types of common features: commitment to perform, ability to perform, activities performed, measurement and analysis, and verifying implementation.

- ***Key Practices***

- The key practices describe the elements of infrastructure and practice that contribute most effectively to the implementation and institutionalization of the area.

Levels of CMM

- ***Level 1:***

- Characterized by period and efforts required by individuals to successfully complete projects.

- ***Level 2:***

- Software project tracking, requirements management, realistic planning, and configuration management processes are in place; successful practices can be repeated.

- ***Level 3:***

- Standard software development and maintenance processes are integrated throughout an organization;
- Software Engineering Process Group is in place to oversee software processes, and training programs are used to ensure understanding and compliance.

Levels of CMM

- *Level 4:*

- Metrics are used to track productivity, processes, and products.
- Project performance is predictable, and quality is consistently high.

- *Level 5:*

- The focus is on continuous process improvement.
- The impact of new processes and technologies can be predicted and effectively implemented when required.

Software Quality Assurance (SQA)

- The process of evaluating the quality of a product and enforcing commitment to software product standards and procedures.
- Software quality assurance (SQA) consists of a means of monitoring the software engineering processes and methods used to ensure quality.
- SQA encompasses the entire software development process, which includes processes such as requirements definition, software design, coding, source code control, code reviews, change management, configuration management, testing, release management, and product integration.

Software Quality Assurance (SQA)

- SQA is organized into goals, commitments, abilities, activities, measurements, and verifications.
- Conformance to software requirements is the foundation from which software quality is measured.
- Specified standards are used to define the development criteria that are used to guide the manner in which software is engineered.
- Software must conform to implicit requirements (ease of use, maintainability, reliability, etc.) as well as its explicit requirements.

SQA Activities

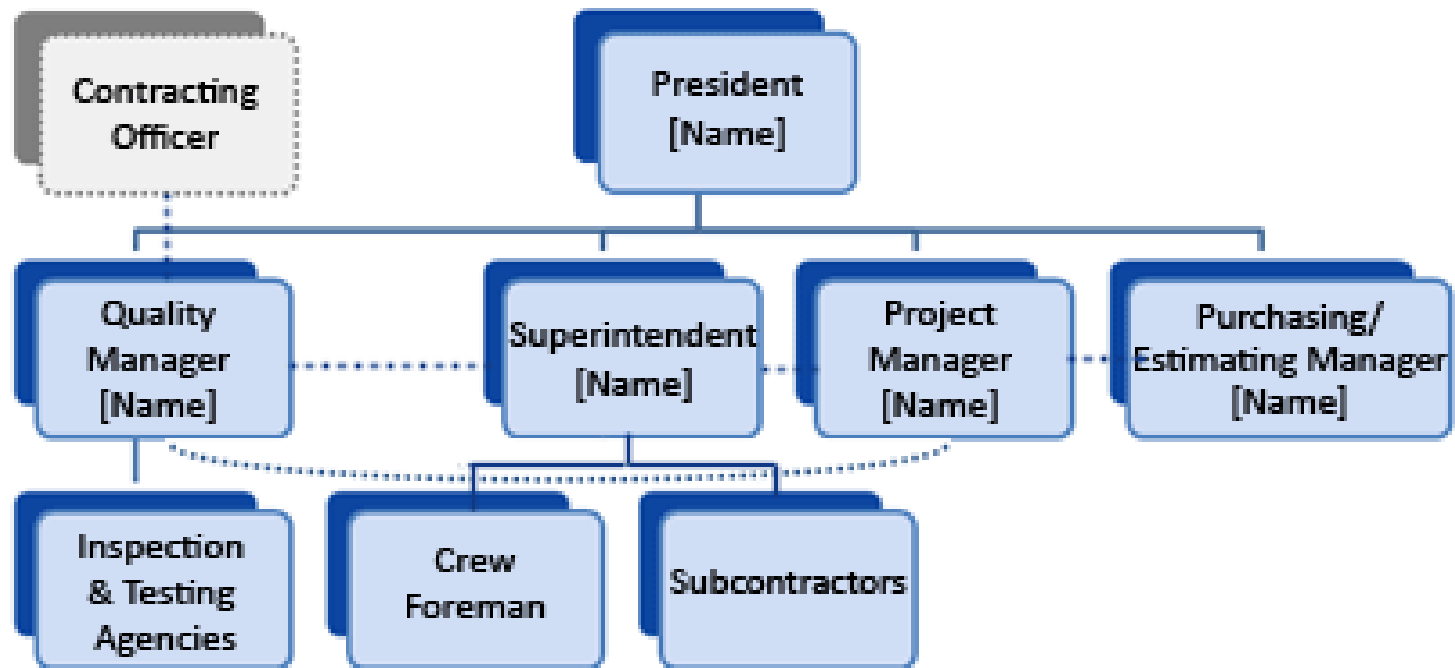
- Formulating a Quality Management Plan
- Applying Software Engineering Techniques
- Conducting Formal Technical Reviews
- Applying a Multi-tiered Testing Strategy
- Enforcing Process Adherence
- Controlling Change
- Measuring Impact of Change
- Performing SQA Audits
- Keeping Records and Reporting

QA Organizational Structure

- The organizational structure has to provide the QA manager with direct organizational paths into every department.
- Small businesses can meet these requirements by assigning the QA responsibilities to someone in management, giving him the authority to manage QA matters throughout the company and creating a QA reporting path to the executive level.
- Employees continue to report to their department manager for disciplinary and non-QA matters, but report to the person responsible for QA on quality questions.

QA Organizational Structure

Project Quality Control Organization Chart



SQA Plan

- A Software Quality Assurance (SQA) plan is a document that outlines the quality assurance strategy and approach for a software development process.
- The SQA plan describes the activities, resources and tools required to ensure that the software product meets the specified quality standards and requirements.
- The SQA typically includes:
 - *SQA Process*
 - *SQA Responsibilities*
 - *SQA Tools*
 - *SQA Deliverables*

SQA Plan

- ***Management Section***

- Describes the place of SQA in the structure of the organization.

- ***Documentation Section***

- Describes each work product produced as part of the software process.

- ***Standards, Practices, and Conventions Section***

- Lists all applicable standards/practices applied during the software process and any metrics to be collected as part of the software engineering work.

SQA Plan

- ***Reviews and Audits Section***
 - Provides an overview of the approach used in the reviews and audits to be conducted during the project
- ***Problem Reporting and Corrective Action Section***
 - Defines procedures for reporting, tracking, and resolving errors or defects, identifies organizational responsibilities for these activities
- ***Other***
 - Tools, SQA methods, change control, record keeping, training, and risk management
- ***Test section***
 - References the test plan and procedure document and defines test record keeping requirements



End of Chapter 8

Thank You !!!!